

Lab 4

Liron Adi 322528787

Shahar Golombek 315099614

מטרת המעבדה:

הכרות עם התוכנה Quartus , ביצוע סינתזה למודלים שיצרנו בעבר (ALU) , ומודלים נוספים שיצרנו במעבדה הנוכחית (PWM) ושימוש בFPGA לצורך בדיקה ויזואלית וחומרית של התוצאות.

בדיקת ביצועים:

כחלק מהמשך העבודה מהמעבדה הראשונה, ביצענו בדיקה שנועדה לבחון כיצד ה ALU-שפיתחנו מתפקד לאחר סינתזה והטמעה על כרטיס FPGA אמיתי. מאחר שהמערכת שתכננו פועלת ללא שעון (א-סינכרונית), נדרשנו להוסיף רגיסטרים סינכרוניים בקצוות המערכת - הן בכניסות והן ביציאות - כדי שנוכל לבצע ניתוח מדויק של זמני הפעולה.

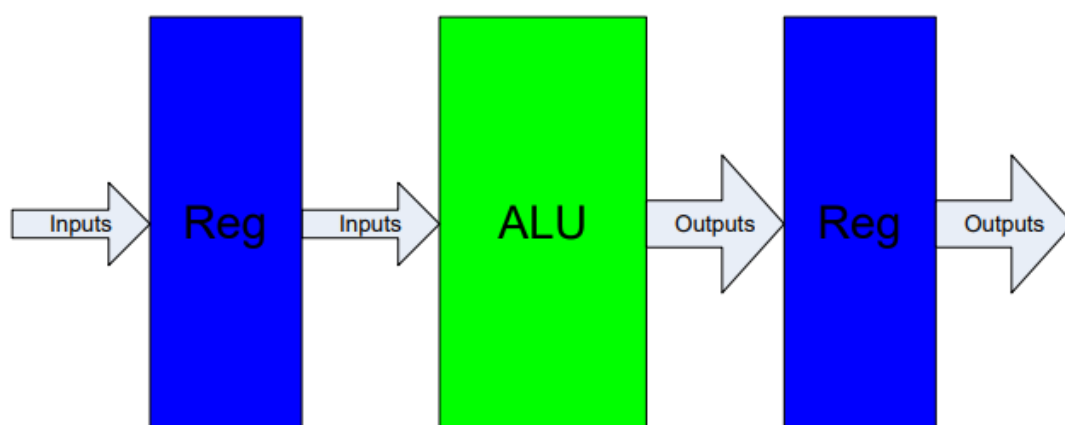


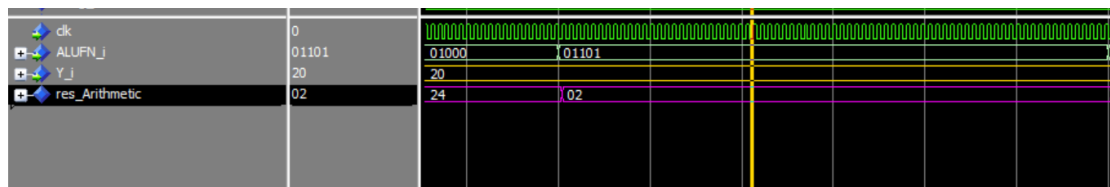
Figure 1 : Test Case in case of pure logic system as ALU

הינה כמה דוגמאות לפעולות הALU שבדקנו תחילה באמצעות ה-ModelSIM, יצרנו TB שבודק כמה פעולות.

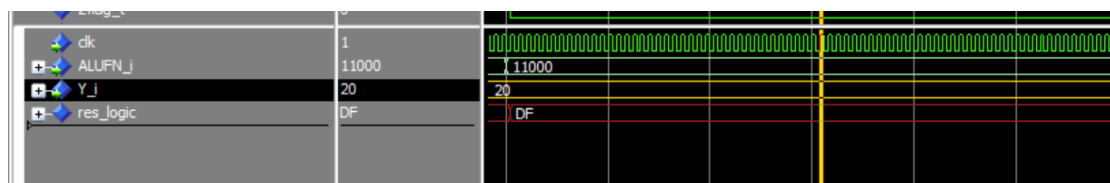
1. הפעולה הראשונה בTEST הייתה פעולת חיבור (ALUFN="01000") בין 20 ב- HEX (=32 בעשרוני) ל- 4. ניתן לראות שהתוצאה היא אכן 20 בHEX.



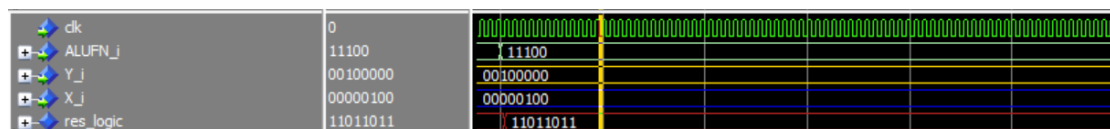
2. פעולת SWAP על Y – מחליפה את ה4 הביט ה LSB עם 4 הביט MSB. בדוגמא הזאת 20 הופך להיות 02.



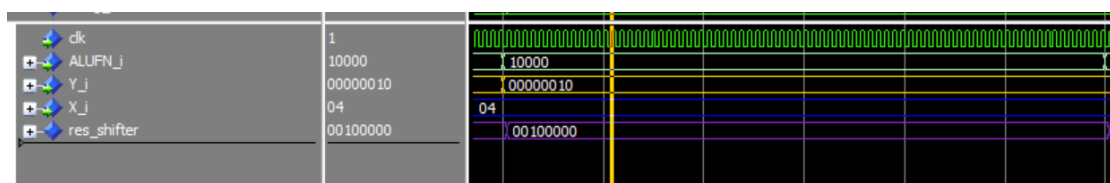
3. פעולת NOT על Y



4. פעולת NOR בין X לY

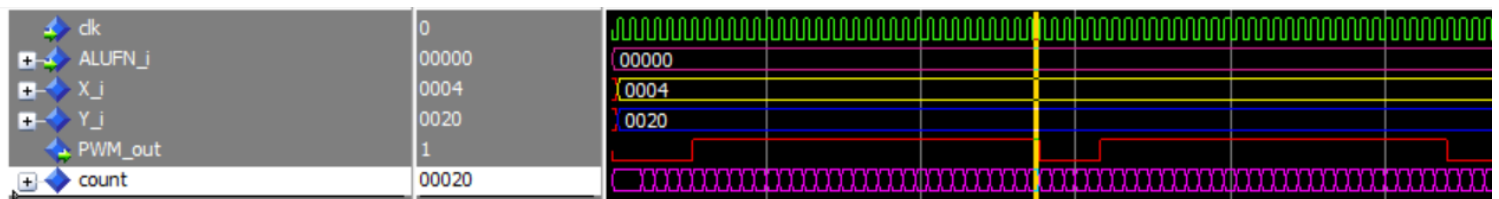


5. פעולת SHIFT LEFT על Y כמות הפעמים שמופיע ב3 הספרות הנמוכות של X (4).

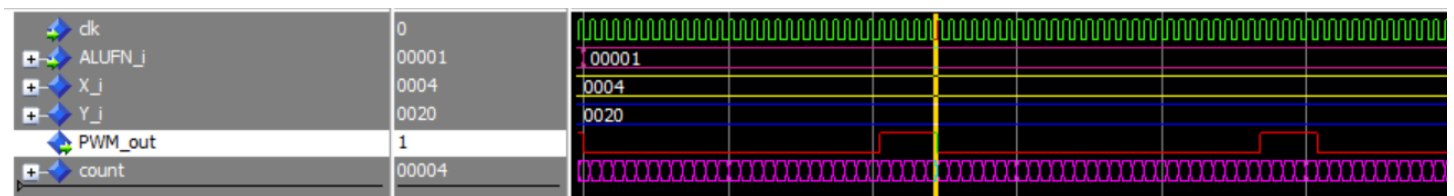


הינה דוגמאות לפעולות PWM שבדקנו עם הTB:

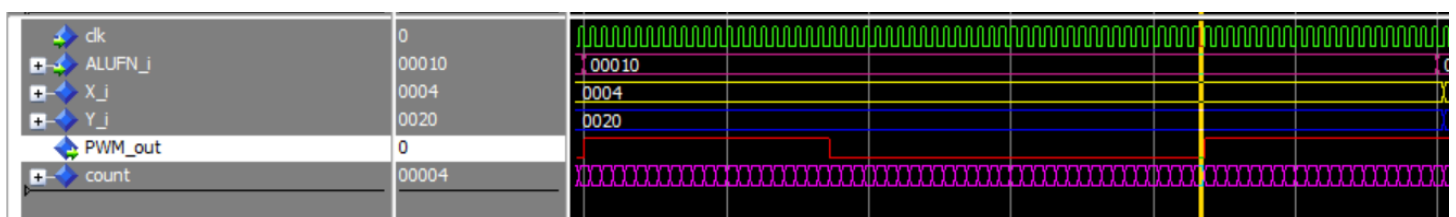
1. Mode0 – ניתן לראות כי עבור מצב זה אות ה-PWM ב- reset 4 מחזורי שעון וב- set 20 HEX מחזורי שעון.



2. Mode1 – ניתן לראות שכאן מתבצעת הפעולה ההפוכה – HEX 20 reset ב- set 4-י מחזורי שעון ב- reset.



3. Mode2 – כאן ניתן לראות שהאות עובר בין set ל- reset בכל פעם שה-counter מגיע ל- 4 (הערך של X) ללא תלות בערך של Y אשר מאפס את ה-counter. מה שיוצר אות PWM עם מחזור 2Y.

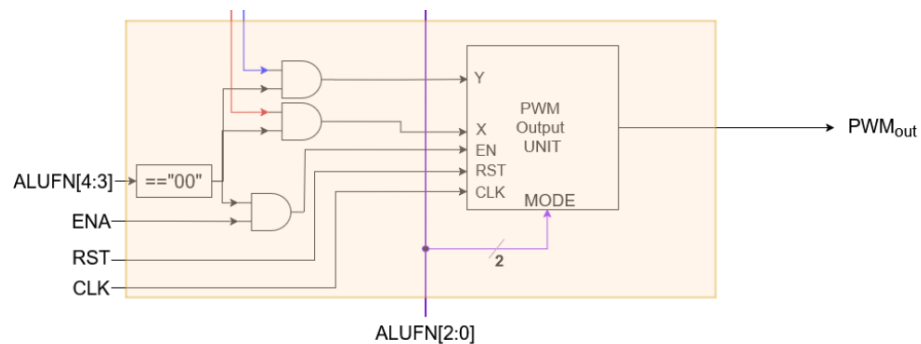


פירוט המערכת:

בסעיף זה נציג את המערכת כולה, תוך התמקדות בתתי-המודולים המרכיבים אותה. עבור כל מודול, נספק סקירה קצרה של אופן פעולתו, נציג את מבנה ה-RTL לאחר סינתזה, נפרט את הלוגיקה שבה הוא עושה שימוש, ונאתר את הנתבי הקריטי שלו. לבסוף, נמחיש את הנתבי הקריטי באמצעות כלי הפיתוח של Quartus.

מערכת PWM:

נממש מערכת שתפקידה ייצור אות PWM עם זמן מחזור וduty cycle שונים לפי תנאים באמצעות ספירה של מחזורי השעון. המערכת תעבוד בהכנסת קוד פעולה שמתחיל ב - "00".



אל המערכת יש 6 כניסות:

- אות כניסה X , ואות כניסה Y – באורך 16 ביט כל אחד.
- שעון
- כניסת Enable
- כניסת Reset
- וקוד הפעולה

המערכת תהיה מסוגלת לעבוד ב3 מצבים שונים כתלות בא X וY:

1. MODE0 – ("000") במצב זה נראה 1 במוצא בהגעה לערך של X ו 0 בהגעה לערך של Y
2. MODE1 – ("001") מצב זה הוא הפוך למצב הקודם.

3. MODE2 – ("010") במצב זה נראה 1 במוצא בהגעה ראשונה ל X ו- 0 במוצא בהגעה נוספת ל X.

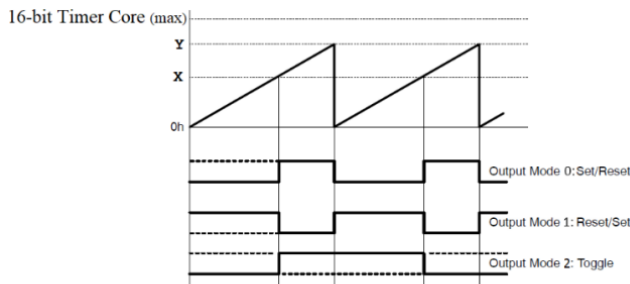
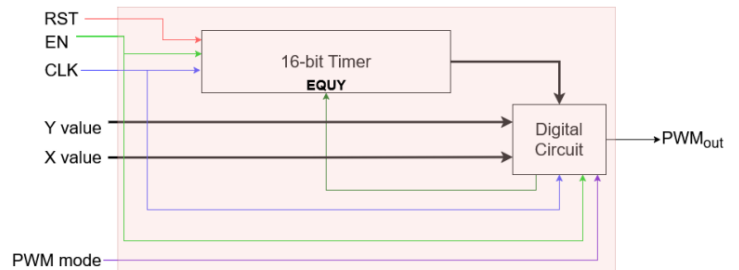


Figure 5: PWM output unit architecture

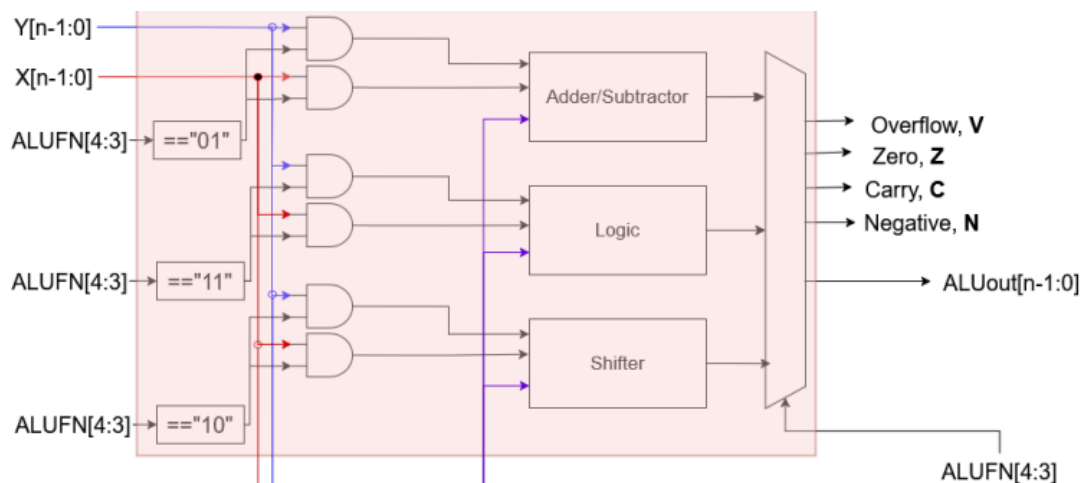
במצב זה יש לאות duty cycle של 50% וזמן מחזור של פעמיים Y.

Figure 4: Digital System subparts architecture



מערכת ALU:

המערכת שברצוננו לממש כוללת מספר מודולים שונים, כאשר כל מודול פועל באופן עצמאי ונפרד מהאחרים. המשתמש בוחר את המודול הרצוי אשר מפעיל את המודול המתאים מתוך המערכת. להלן תיאור המערכת אותה נממש:



מערכת זו מקבלת 3 כניסות:

- אות כניסה X
- אות כניסה Y
- קוד פעולה – שמסייע בהחלטה איזה סוג מודול נשתמש ואיזה פעולה בדיוק תתבצע.

שלושת המודולים:

- Arithmetic – ("01") הוא אחראי על ביצוע פעולה חשבונית פשוטות ועל פעולת swap.
- Shifter – ("10") אחראי על ביצוע הזזה למספר ימינה או שמאלה

- Logic – ("11") אחראי על ביצוע פעולות בוליאניות

מוצא המערכת:

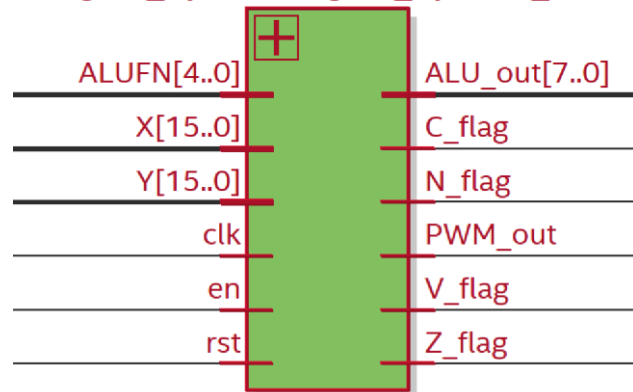
- פלט תוצאת החישוב של המודול בו השתמשנו
- 4 דגלים - Z (Zero), N (Negative), C (Carry), V (Overflow)

סיכום המערכת:

את שני המודלים נעטוף בשכבה עליונה (digital system) שתנהל את כל המערכת.
על מנת לעבוד עם הFPGA ניצור שכבה נוספת שנקראת GPIO שתהיה אחראית על תצוגה חומריתת וחיבורים של המערכת.

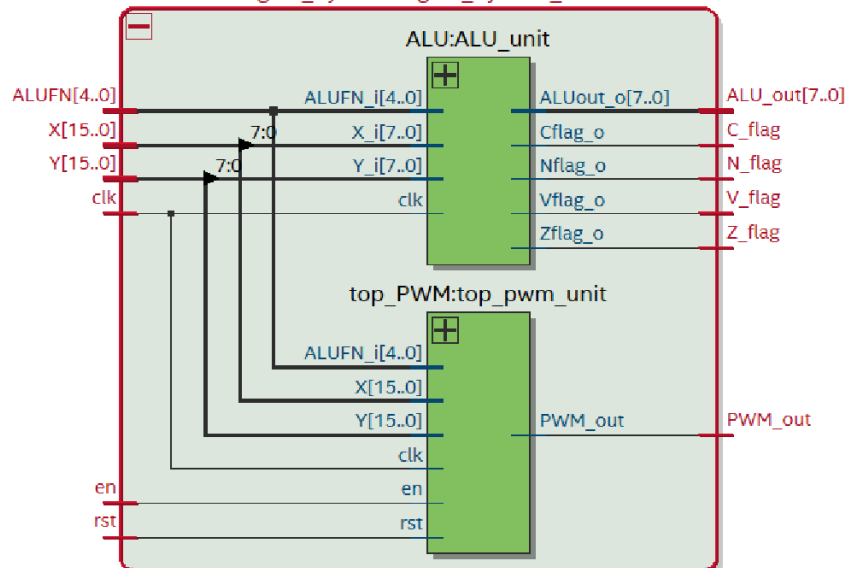
שרטוט ENTITY של המערכת כולה:

Digital_System:Digital_System_unit

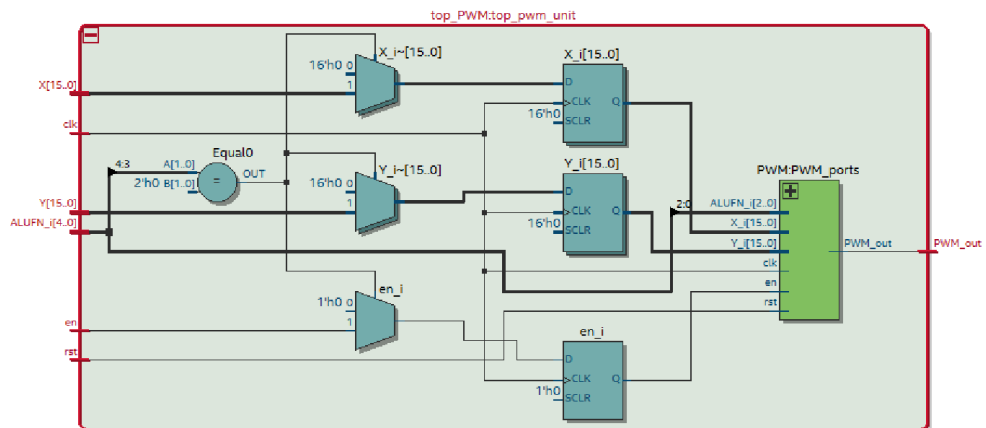


שרטוט RTL של המערכת כולה:

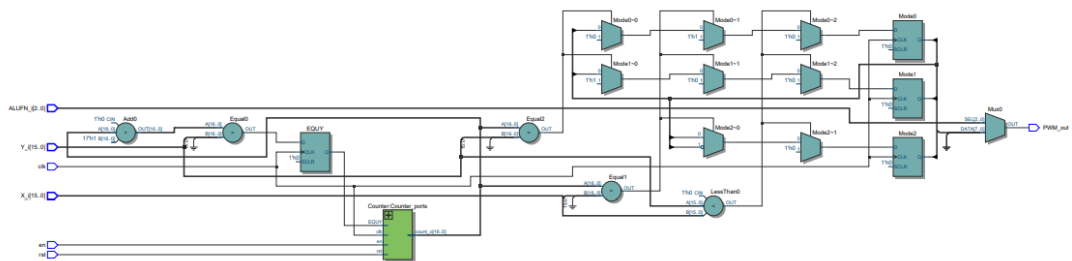
Digital_System:Digital_System_unit



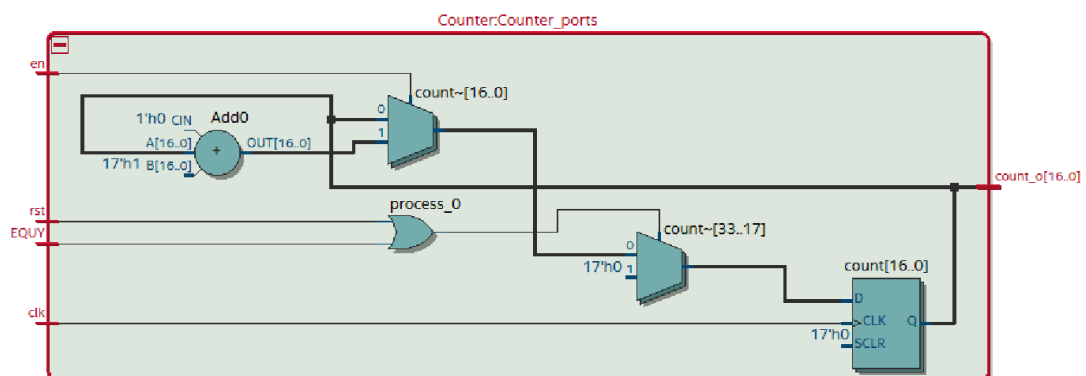
שרטוט RTL של מעטפת המודול PWM:



שרטוט RTL של המודול PWM עצמו:



שרטוט RTL של הCounter :

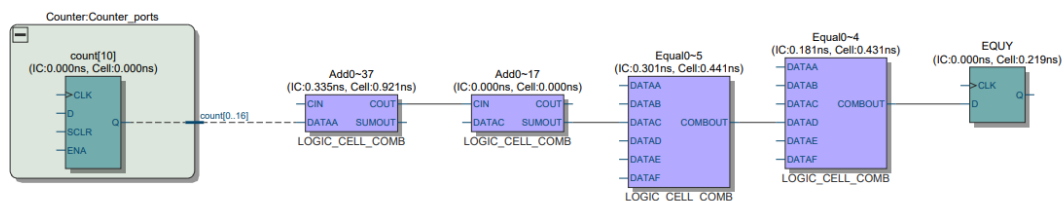


במסגרת העבודה עם לוגיקה דיגיטלית, אנו מפתחים קוד המתבסס על אלמנטים לוגיים הניתנים לקינפוג בהתאם לתיאור ההתנהגותי שנכתב – תהליך המבוצע במהלך הסינתזה. בחלק זה נציג את הלוגיקה שמאחורי כל אחד מהמודולים, בהתאם לדרישות ולמבנה הפנימי שלהם:

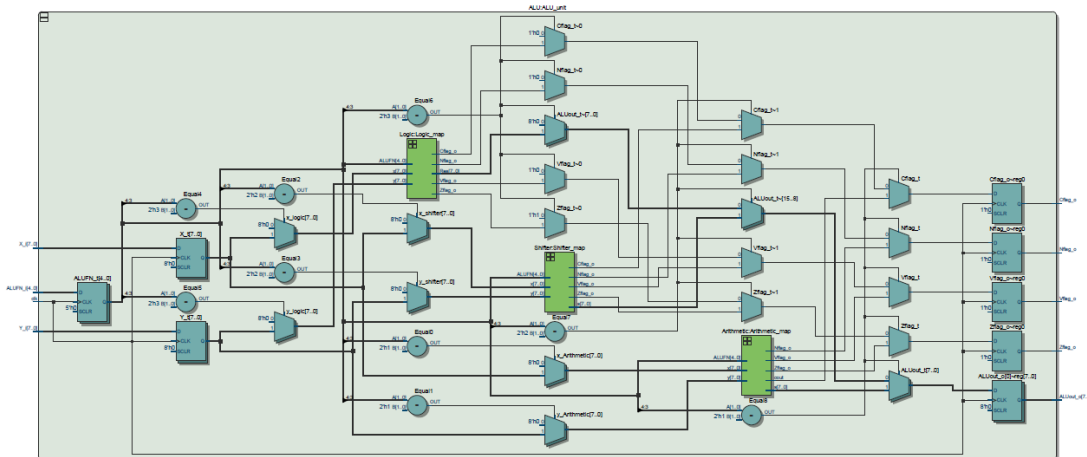
	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	51
2		
3	▼ Combinational ALUT usage for logic	76
1	-- 7 input functions	0
2	-- 6 input functions	26
3	-- 5 input functions	5
4	-- 4 input functions	5
5	-- <=3 input functions	40
4		
5	Dedicated logic registers	21
6		
7	I/O pins	39
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	clk~input
12	Maximum fan-out	21
13	Total fan-out	398
14	Average fan-out	2.27

נתיב קריטי

הנתיב הקריטי מהווה מרכיב מרכזי בהבנת זרימת המידע בתוך המערכת. מאחר שכל רכיב, גם כאשר הוא פועל במקביל לאחרים, כולל זמן השהייה פנימי, יש לקחת בחשבון את משך הזמן הכולל הדרוש להתייצבות המערכת לפני שניתן לעבד קלט חדש. להלן ניתוח ואיתור הנתיב הקריטי במודול הנוכחי:



שרטוט RTL של ALU:

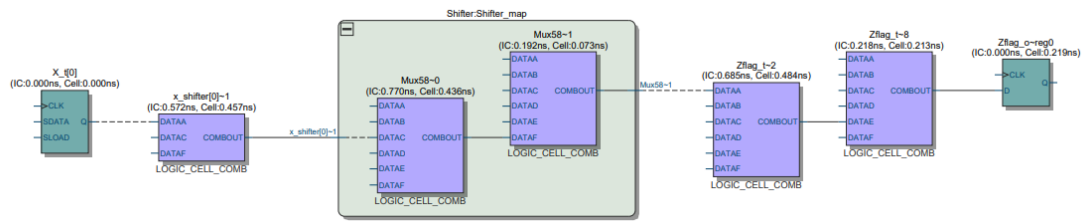


במסגרת העבודה עם לוגיקה דיגיטלית, אנו מפתחים קוד המתבסס על אלמנטים לוגיים הניתנים לקינפוג בהתאם לתיאור ההתנהגותי שנכתב – תהליך המבוצע במהלך הסינתזה. בחלק זה נציג את הלוגיקה שמאחורי כל אחד מהמודולים, בהתאם לדרישות ולמבנה הפנימי שלהם:

<<Filter>>		
	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	133
2		
3	▼ Combinational ALUT usage for logic	161
1	-- 7 input functions	3
2	-- 6 input functions	99
3	-- 5 input functions	14
4	-- 4 input functions	12
5	-- <=3 input functions	33
4		
5	Dedicated logic registers	33
6		
7	I/O pins	34
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	ALUFN_t[0]
12	Maximum fan-out	74
13	Total fan-out	929
14	Average fan-out	3.55

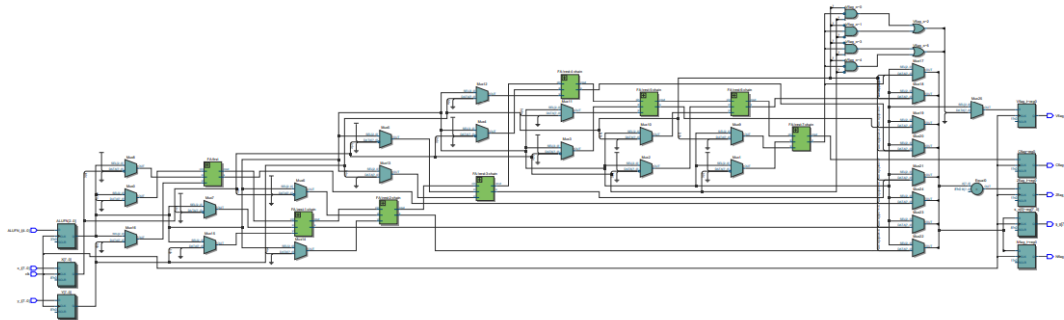
נתיב קריטי

הנתיב הקריטי מהווה מרכיב מרכזי בהבנת זרימת המידע בתוך המערכת. מאחר שכל רכיב, גם כאשר הוא פועל במקביל לאחרים, כולל זמן השהייה פנימי, יש לקחת בחשבון את משך הזמן הכולל הדרוש להתייצבות המערכת לפני שניתן לעבד קלט חדש. להלן ניתוח ואיתור הנתיב הקריטי במודול הנוכחי:



מצאי הניתוח מראים שהנתיב הקריטי ביותר עובר דרך מודול ה-Shifter, כפי שצפינו מראש, וזאת בשל מורכבות הלוגיקה הרבה שהוא כולל ביחס למודולים האחרים.

שרטוט RTL arithmetic:

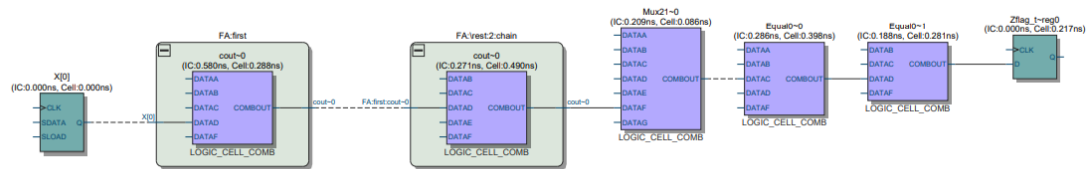


במסגרת העבודה עם לוגיקה דיגיטלית, אנו מפתחים קוד המתבסס על אלמנטים לוגיים הניתנים לקינפוג בהתאם לתיאור ההתנהגותי שנכתב – תהליך המבוצע במהלך הסינתזה. בחלק זה נציג את הלוגיקה שמאחורי כל אחד מהמודולים, בהתאם לדרישות ולמבנה הפנימי שלהם:

	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	24
2		
3	▼ Combinational ALUT usage for logic	35
1	-- 7 input functions	1
2	-- 6 input functions	7
3	-- 5 input functions	11
4	-- 4 input functions	14
5	-- <=3 input functions	2
4		
5	Dedicated logic registers	31
6		
7	I/O pins	34
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	clk~input
12	Maximum fan-out	31
13	Total fan-out	274
14	Average fan-out	2.04

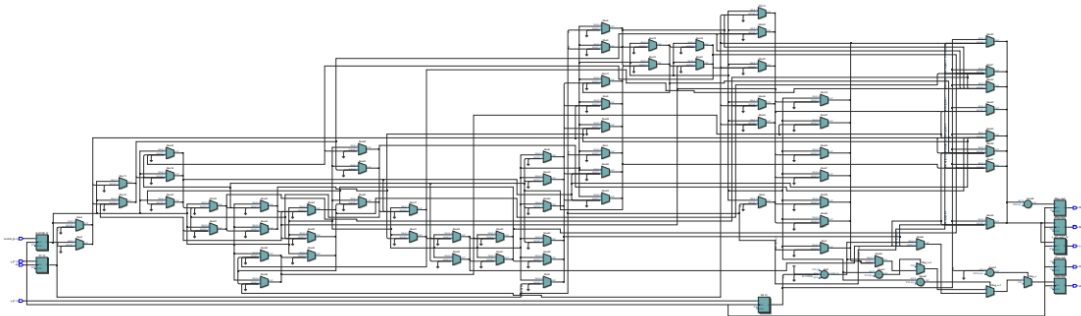
נתיב קריטי

הנתיב הקריטי מהווה מרכיב מרכזי בהבנת זרימת המידע בתוך המערכת. מאחר שכל רכיב, גם כאשר הוא פועל במקביל לאחרים, כולל זמן השהייה פנימי, יש לקחת בחשבון את משך הזמן הכולל הדרוש להתייצבות המערכת לפני שניתן לעבד קלט חדש. להלן ניתוח ואיתור הנתיב הקריטי במודול הנוכחי:



נשים לב שהמסלול הקריטי עובר דרך הFA, מה שהמסתדר אם מה שהנחנו כי FA פועל ביט ביט.

שרטוט RTL של Shifter :

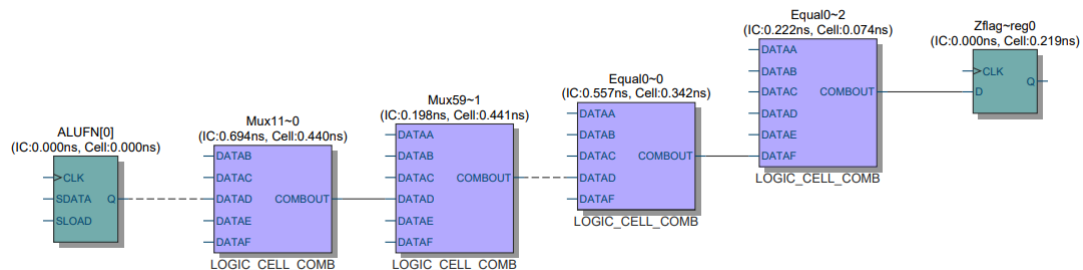


במסגרת העבודה עם לוגיקה דיגיטלית, אנו מפתחים קוד המתבסס על אלמנטים לוגיים הניתנים לקיפוג בהתאם לתיאור ההתנהגותי שנכתב – תהליך המבוצע במהלך הסינתזה. בחלק זה נציג את הלוגיקה שמאחורי כל אחד מהמודולים, בהתאם לדרישות ולמבנה הפנימי שלהם:

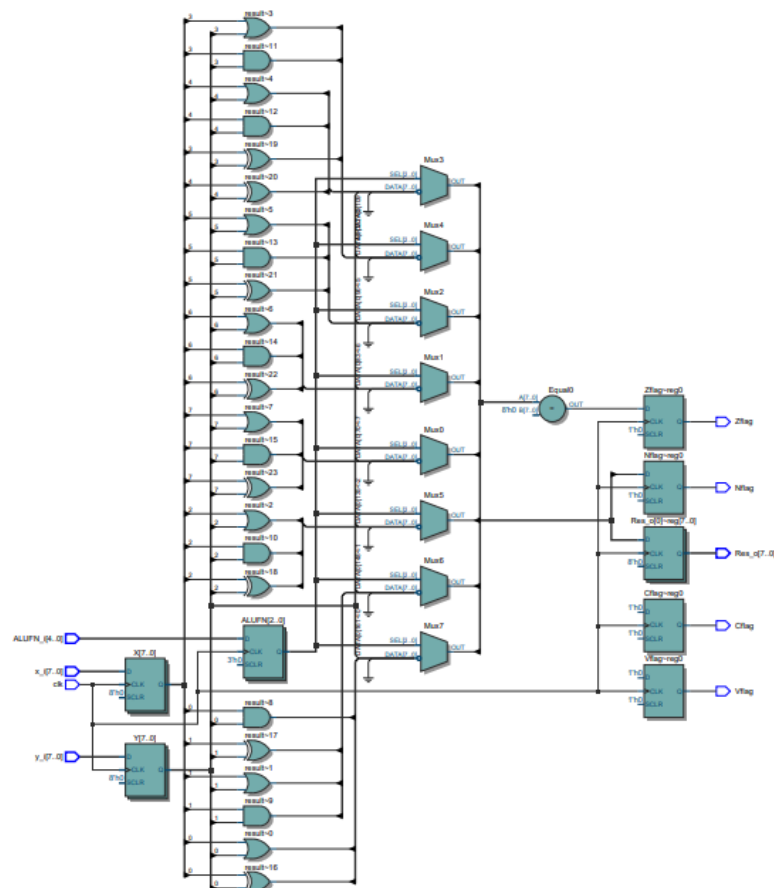
	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	46
2		
3	▼ Combinational ALUT usage for logic	66
1	-- 7 input functions	1
2	-- 6 input functions	23
3	-- 5 input functions	14
4	-- 4 input functions	19
5	-- <=3 input functions	9
4		
5	Dedicated logic registers	25
6		
7	I/O pins	34
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	ALUFN[0]
12	Maximum fan-out	35
13	Total fan-out	411
14	Average fan-out	2.58

נתיב קריטי

הנתיב הקריטי מהווה מרכיב מרכזי בהבנת זרימת המידע בתוך המערכת. מאחר שכל רכיב, גם כאשר הוא פועל במקביל לאחרים, כולל זמן שהייה פנימי, יש לקחת בחשבון את משך הזמן הכולל הדרוש להתייצבות המערכת לפני שניתן לעבד קלט חדש. להלן ניתוח ואיתור הנתיב הקריטי במודול הנוכחי:



שרטוט RTL של Logic :

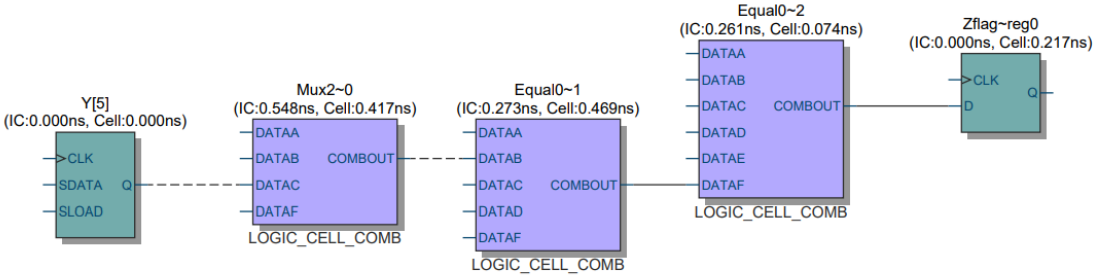


במסגרת העבודה עם לוגיקה דיגיטלית, אנו מפתחים קוד המתבסס על אלמנטים לוגיים הניתנים לקינפוג בהתאם לתיאור ההתנהגותי שנכתב – תהליך המבוצע במהלך הסינתזה. בחלק זה נציג את הלוגיקה שמאחורי כל אחד מהמודולים, בהתאם לדרישות ולמבנה הפנימי שלהם:

	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	19
2		
3	▼ Combinational ALUT usage for logic	19
1	-- 7 input functions	0
2	-- 6 input functions	1
3	-- 5 input functions	10
4	-- 4 input functions	4
5	-- <=3 input functions	4
4		
5	Dedicated logic registers	29
6		
7	I/O pins	34
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	clk~input
12	Maximum fan-out	29
13	Total fan-out	186
14	Average fan-out	1.60

נתיב קריטי

הנתיב הקריטי מהווה מרכיב מרכזי בהבנת זרימת המידע בתוך המערכת. מאחר שכל רכיב, גם כאשר הוא פועל במקביל לאחרים, כולל זמן השהייה פנימי, יש לקחת בחשבון את משך הזמן הכולל הדרוש להתייצבות המערכת לפני שניתן לעבד קלט חדש. להלן ניתוח ואיתור הנתיב הקריטי במודול הנוכחי:



אופטימיזציה חומרית

במהלך שלב האופטימיזציה של המערכת, הוספנו את מודול ה-PLL הזמין על גבי כרטיס ה-FPGA במטרה להעלות את תדר העבודה של המערכת – ובכך גם את התדר המרבי האפשרי. כפי שתואר בשלבים הקודמים של מציאת התדר המקסימלי, שילוב ה-PLL הוביל לעלייה בתדר לכ-60MHz.

נזכיר כי הגדרנו את זמן המחזור בקובץ ה-SDC כ-20ns, כלומר תדר של $50\text{MHz} \cdot \frac{1}{20} = 100\text{MHz}$. לאחר ההגדרה, קבענו את תדר המוצא של ה-PLL להיות 100MHz, שידכנו לשעוני המערכת השונים.

Signal Tap

לצורך ביצוע ווריקציה של החומרה בפועל, השתמשנו בכלי **Signal Tap** המובנה בסביבת הפיתוח Quartus. הכלי מאפשר ללכוד את מצב הסיגנלים בזמן אמת מתוך הכרטיס עצמו, ולבחון את התנהגות המערכת במהלך הריצה.

בחרנו לעקוב אחר סיגנלים מרכזיים במערכת, כאשר תנאי ההפעלה ללכידה היה מבוסס על falling edge מכיוון שהKEY0 הם כפתורי pull down ונרצה לראות את השינוי במסך באותו זמן שנראה את השינוי בFPJA כלומר הלכידה תתבצע בירידת מתח. בנוסף, בחרנו להשתמש בתנאי לכידה מסוג **Basic OR**, כך שכל שינוי באחד מהסיגנלים הנבחרים יספיק כדי להפעיל את הלכידה.

בתוך הלכידה נדפיס למסך את ערכי הכניסות והפלטים של המערכת, על מנת לאמת את תקינות פעולתה בפועל בהתאם למצופה.

	☒	KEY0_3[3..0]	☒	☒	XXXX (OK)
	☒	KEY0_3[3]	☒	☒	
	☒	KEY0_3[2]	☒	☒	
	☒	KEY0_3[1]	☒	☒	
	☒	KEY0_3[0]	☒	☒	

בתמונה הבאה ניתן להבחין שהגדרתי את הקוד פעולה 08 עובר ALUFN שזה שווה ערך ל"01000" שזהו הקוד פעולה של פעולת החיבור:

	☒	Y_LOW_8BIT[7..0]	00h
	☒	X_LOW_8BIT[7..0]	00h
	☒	Y_HIGH_8BIT[7..0]	00h
	☒	X_HIGH_8BIT[7..0]	00h
	☒	...stem:Digital_System_unit X[15..0]	0000h
	☒	...stem:Digital_System_unit Y[15..0]	0000h
	☒	...Digital_System_unit ALU_out[7..0]	00h
	☒	...Digital_System_unit ALUFN[4..0]	00h 08h

כעת ניתן לראות שהכנסתי לרגיסטר Y את הערך 1 ומיד קיבלנו את עידכון חיבור ה-ALU ל0 מכיוון שהתחלתי את כל המערכת על אפסים:

Type	Alias	Name	-512	-256	0	256	512	768	1024	1280	1536	1792	2048	2304	2560
		Y_LOW_8BIT[7..0]	00h			01h									
		X_LOW_8BIT[7..0]				00h									
		Y_HIGH_8BIT[7..0]				00h									
		X_HIGH_8BIT[7..0]				00h									
		...stem:Digital_System_unit X[15..0]				0000h									
		...stem:Digital_System_unit Y[15..0]	0000h			0001h									
		...:Digital_System_unit ALU_out[7..0]	00h			01h									
		...:Digital_System_unit ALUFN[4..0]				08h									

ולבסוף הגדרתי את X להיות 4 וקיבלנו את תוצאת החיבור 5:

Type	Alias	Name	-512	-256	0	256	512	768	1024	1280	1536	1792	2048	2304	2560	2816	3072	3328	3584			
		Y_LOW_8BIT[7..0]											01h									
		X_LOW_8BIT[7..0]	00h					04h														
		Y_HIGH_8BIT[7..0]											00h									
		X_HIGH_8BIT[7..0]											00h									
		...stem:Digital_System_unit X[15..0]	0000h					0004h														
		...stem:Digital_System_unit Y[15..0]											0001h									
		...:Digital_System_unit ALU_out[7..0]	01h					05h														
		...:Digital_System_unit ALUFN[4..0]											08h									